

DATABASE DESIGN DOCUMENT

QR-Based Visitor Management System (VMS)

Version 2.0

Submitted for Internship Evaluation
April 2026

Field	Details
Document Type	Database Design Documentation
System Name	QR-Based Visitor Management System
Version	v2.0.0
Prepared By	Amit Kushwaha
Department	MP Online Limited
Date	27 April 2026
Project Type	Internship Project

1. Project Overview

The QR-Based Visitor Management System (VMS) is a full-stack web application developed as part of an internship project. It digitizes visitor entry and exit for organizations with single or multiple branches.

Key capabilities:

- Visitor registration with photo capture and QR code generation
- Automated email dispatch of QR codes to visitors
- Real-time QR scanning for check-in and check-out
- Role-based access control (Guard, Admin, Super Admin)
- Multi-branch support with centralized dashboard
- Comprehensive audit logging of all system actions

2. Entity-Relationship (ER) Diagram

The ER diagram shows all five entities in the VMS database, their attributes, primary keys (PK), foreign keys (FK), and the relationships between them. Lines connecting entities indicate foreign key references; cardinality labels (1:N) show one-to-many relationships.

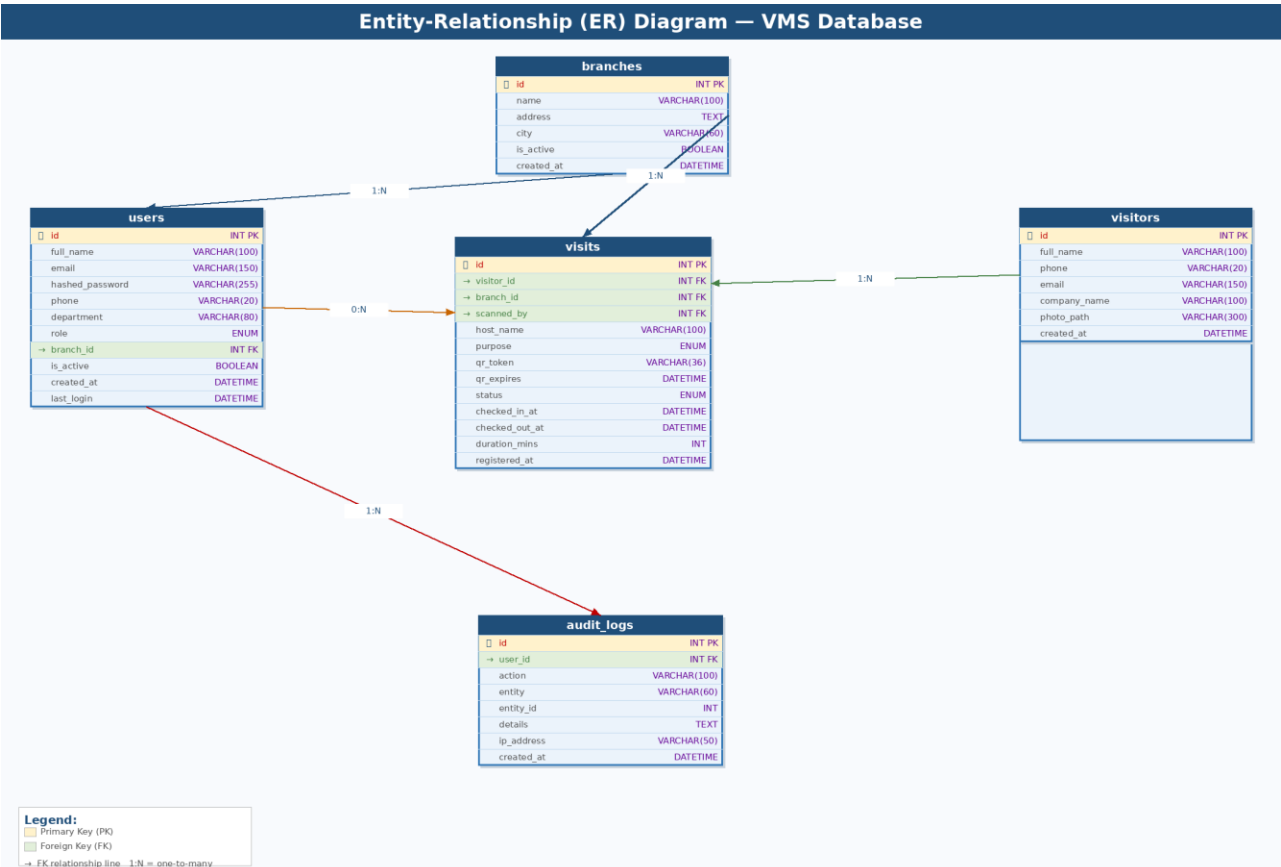


Figure 1: ER Diagram — VMS Database Schema

Key observations from the ER diagram:

- branches is the root entity — both users and visits reference it
- visitors and visits are separate entities to support repeat visitors without data duplication
- visits is the central table with three foreign keys (visitor, branch, scanned_by)
- audit_logs references users to track who performed each action

3. Data Flow Diagram (DFD) — Level 1

The DFD Level 1 shows how data flows between external entities (Visitor, Guard, Admin, Super Admin), internal processes, and data stores. It represents the system from a data movement perspective rather than a control flow perspective.

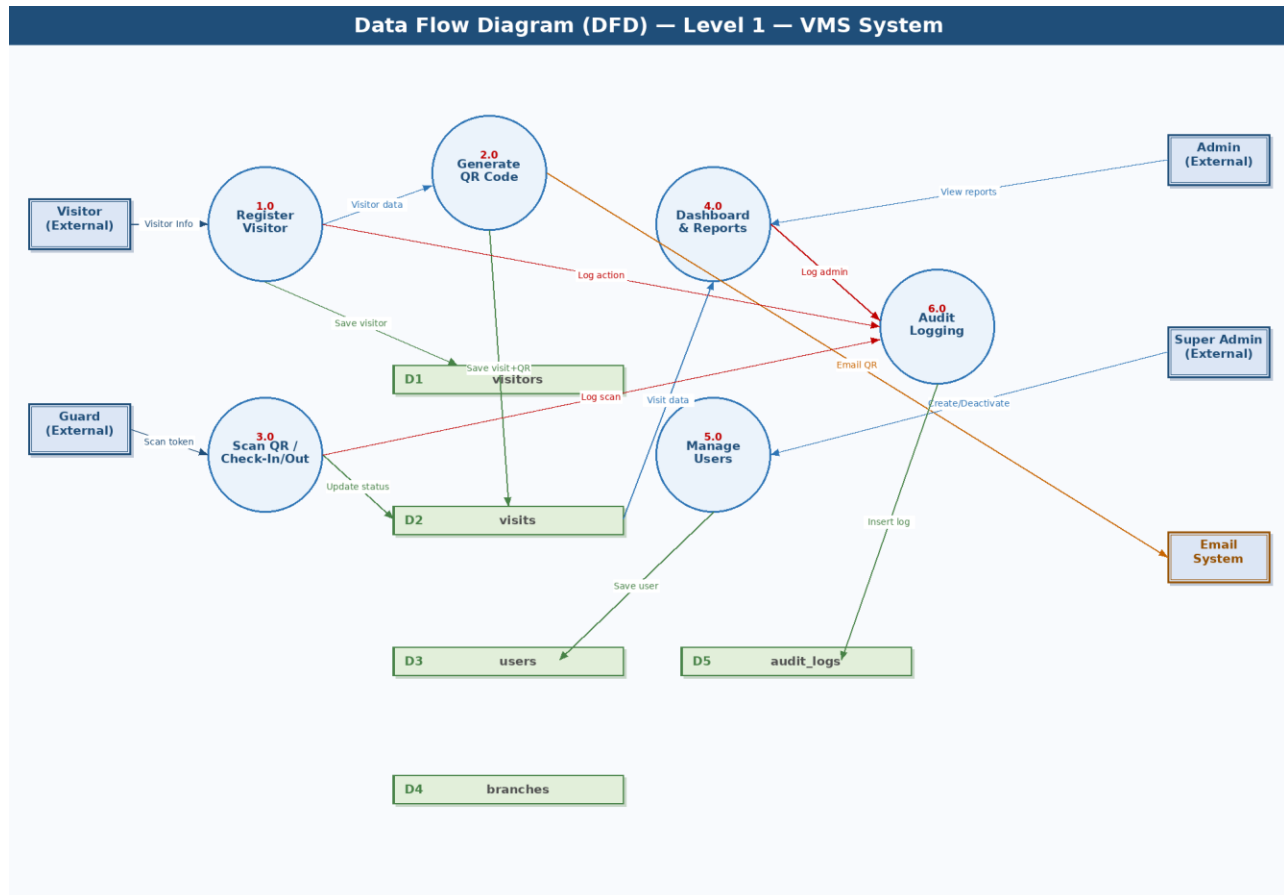


Figure 2: Data Flow Diagram (DFD) Level 1 — VMS System

DFD Symbols used:

- Rectangle with double border = External Entity (outside the system boundary)
- Circle = Process (data transformation step)
- Open rectangle = Data Store (persisted data — maps to a database table)
- Arrows = Data Flows (named data moving between elements)

Main data flows:

- Visitor submits info → Process 1.0 registers visitor → saves to visitors store
- Process 2.0 generates QR → saves to visits store → emails QR to visitor
- Guard scans QR → Process 3.0 updates visit status in visits store
- Admin views dashboard via Process 4.0 which reads from visits store
- All processes feed into Process 6.0 (Audit Logging) → audit_logs store

4. Database Structure Diagram

The database structure diagram shows the physical schema — all tables with their columns, data types, constraints, and the foreign key relationships connecting them. This is the closest representation to what is actually created in the database.

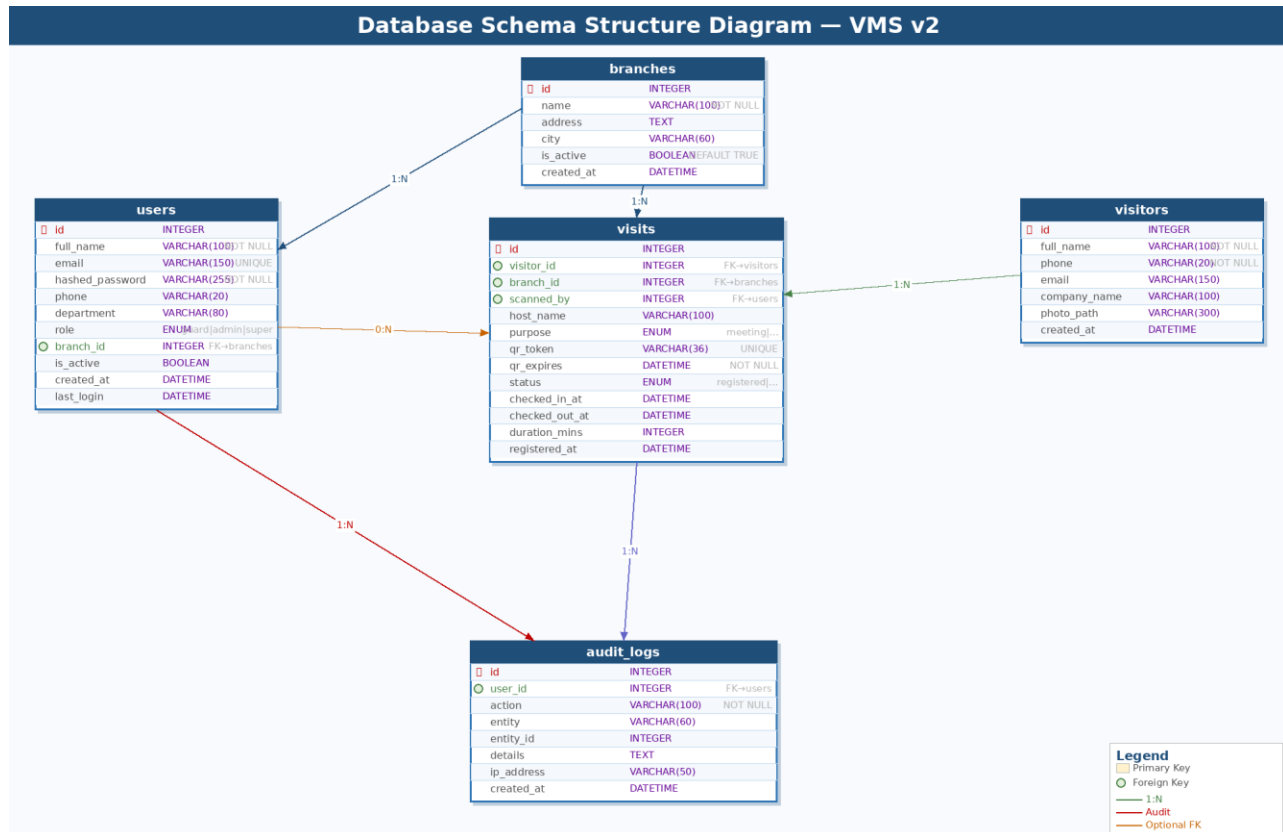


Figure 3: Database Schema Structure Diagram — All 5 Tables

The diagram uses the following visual conventions:

- Yellow rows = Primary Key (PK) columns
- Green rows = Foreign Key (FK) columns with reference indicated
- Purple text = Data type of the column
- Arrows between tables = foreign key relationships with cardinality

5. Database Tables — Detailed Definitions

This section provides the complete column-level specification for each table.

5.1 branches

Stores information about office branches.

Column Name	Data Type	Constraint	Description
id	INTEGER	PK, AUTO_INCREMENT	Unique branch identifier
name	VARCHAR(100)	NOT NULL	Branch/office name
address	TEXT	NULLABLE	Full address of the branch
city	VARCHAR(60)	NULLABLE	City where branch is located
is_active	BOOLEAN	DEFAULT TRUE	Whether branch is currently active
created_at	DATETIME	DEFAULT NOW()	Timestamp when branch was added

5.2 users

System users — guards, admins, and super admins — who operate the VMS.

Column Name	Data Type	Constraint	Description
id	INTEGER	PK, AUTO_INCREMENT	Unique user identifier
full_name	VARCHAR(100)	NOT NULL	User's full name
email	VARCHAR(150)	UNIQUE, NOT NULL	Login email address (indexed)
hashed_password	VARCHAR(255)	NOT NULL	bcrypt hashed password
phone	VARCHAR(20)	NULLABLE	Contact phone number
department	VARCHAR(80)	NULLABLE	Department name
role	ENUM	NOT NULL	Role: guard / admin / super_admin
branch_id	INTEGER	FK → branches.id	Branch this user belongs to
is_active	BOOLEAN	DEFAULT TRUE	Account active/inactive flag
created_at	DATETIME	DEFAULT NOW()	Account creation timestamp
last_login	DATETIME	NULLABLE	Last successful login timestamp

5.3 visitors

Profile records for people who visit the organization.

Column Name	Data Type	Constraint	Description
id	INTEGER	PK, AUTO_INCREMENT	Unique visitor identifier
full_name	VARCHAR(100)	NOT NULL	Visitor's full name
phone	VARCHAR(20)	NOT NULL	Visitor's mobile number
email	VARCHAR(150)	NULLABLE	Email (used to send QR code)
company_name	VARCHAR(100)	NULLABLE	Visitor's organization name
photo_path	VARCHAR(300)	NULLABLE	Relative path to visitor's photo file
created_at	DATETIME	DEFAULT NOW()	Visitor record creation timestamp

5.4 visits

Central table — each row is one visit event, storing the full lifecycle from QR generation to check-out.

Column Name	Data Type	Constraint	Description
id	INTEGER	PK, AUTO_INCREMENT	Unique visit identifier
visitor_id	INTEGER	FK → visitors.id	Reference to the visitor
branch_id	INTEGER	FK → branches.id	Branch where visit is scheduled
scanned_by	INTEGER	FK → users.id, NULL	Guard/user who scanned the QR
host_name	VARCHAR(100)	NULLABLE	Name of the person being visited
purpose	ENUM	NOT NULL	meeting / delivery / interview / maintenance / other
notes	TEXT	NULLABLE	Additional visit notes
qr_token	VARCHAR(36)	UNIQUE, NOT NULL	UUID token embedded in QR code (indexed)
qr_image	TEXT	NULLABLE	Base64 encoded PNG image of QR code
qr_expires	DATETIME	NOT NULL	QR code expiry timestamp
qr_expiry_hours	INTEGER	NULLABLE	Hours configured for QR expiry (for audit)
status	ENUM	NOT NULL	registered / checked_in / checked_out / expired
checked_in_at	DATETIME	NULLABLE	Timestamp when visitor checked in

Column Name	Data Type	Constraint	Description
checked_out_at	DATETIME	NULLABLE	Timestamp when visitor checked out
duration_mins	INTEGER	NULLABLE	Duration of visit in minutes
registered_at	DATETIME	DEFAULT NOW()	Timestamp of visit registration

5.5 audit_logs

Immutable log of all significant system actions for super admin review.

Column Name	Data Type	Constraint	Description
id	INTEGER	PK, AUTO_INCREMENT	Unique audit log entry ID
user_id	INTEGER	FK → users.id, NULL	User who performed the action
action	VARCHAR(100)	NOT NULL	Action performed (e.g., visitor_registered)
entity	VARCHAR(60)	NULLABLE	Entity type affected (e.g., Visit, User)
entity_id	INTEGER	NULLABLE	ID of the affected entity
details	TEXT	NULLABLE	JSON or text details about the action
ip_address	VARCHAR(50)	NULLABLE	IP address of client
created_at	DATETIME	DEFAULT NOW()	Timestamp of audit event

6. Table Relationships

Foreign Key	References	Relationship	Description
users.branch_id	branches.id	Many-to-One	Many users belong to one branch
visits.visitor_id	visitors.id	Many-to-One	A visitor can have multiple visits
visits.branch_id	branches.id	Many-to-One	Multiple visits can occur at one branch
visits.scanned_by	users.id	Many-to-One (Nullable)	A guard/user scans the QR for check-in/out
audit_logs.user_id	users.id	Many-to-One (Nullable)	Tracks which user performed each action

7. Enumerated Types

Enum Type	Value	Meaning
UserRole	guard	Can only scan QR codes for check-in/out
UserRole	admin	Access to dashboard and visitor management
UserRole	super_admin	Full system access including users, branches, audit
VisitPurpose	meeting	Visitor is attending a scheduled meeting
VisitPurpose	delivery	Visitor is delivering goods or documents
VisitPurpose	interview	Visitor is appearing for an interview
VisitPurpose	maintenance	Visitor is for maintenance/repair work
VisitPurpose	other	Any other purpose not listed above
VisitStatus	registered	QR code generated, visitor not yet arrived
VisitStatus	checked_in	Visitor has entered the building
VisitStatus	checked_out	Visitor has left the building
VisitStatus	expired	QR expired; visitor never showed up

8. Indexing Strategy

Table	Indexed Column	Reason
users	email	Login lookup — queried on every authentication request
visits	qr_token	QR scan — matched on every check-in/check-out event
branches	id (PK)	Joined by users and visits frequently
visitors	id (PK)	Joined by visits for visitor details
audit_logs	created_at (implicit)	Sorted for audit trail display

9. Design Decisions & Rationale

9.1 Visitors and Visits are Separate Tables

Separating visitor identity from visit events follows normalization. A visitor who comes multiple times appears only once in visitors, with multiple rows in visits. This avoids duplication and supports visit history.

9.2 QR Token Stored as UUID String

The `qr_token` is a UUID (36-character string), not an integer. This prevents sequential guessing attacks and ensures global uniqueness.

9.3 QR Image Stored as Base64 in Database

The `qr_image` column stores the QR as a base64 data URI. This simplifies retrieval (no file system lookup) at the cost of larger row size — acceptable for office-scale traffic.

9.4 Audit Log is Append-Only

The `audit_logs` table has no update or delete routes. Records are only inserted, ensuring a tamper-resistant audit trail.

9.5 Role-Based Access via ENUM

User roles are stored as an ENUM enforced at the database level, allowing efficient role-based access checks in the backend.

10. Summary

The VMS database consists of 5 tables with well-defined relationships, enforced constraints, and strategic indexing. The design prioritizes:

- Data integrity through foreign keys and NOT NULL constraints
- Security through UUID tokens and hashed passwords
- Scalability through normalized visitor/visit separation
- Auditability through an immutable audit log
- Simplicity through clear enum-based status and role management

This document — including the ER diagram, DFD, and schema structure diagram — serves as the complete technical reference for the database layer of the VMS internship project.